

Program Representations

Java Bytecode

Announcements

- HW1 due Friday: Ad-hoc testing
- You should all have goldengate accounts!

Agenda

- Unix review
- Java Bytecode Part 2

Unix!

pwd command

The first command we'll look at is `pwd`. This command prints the **path** to the **working directory**.

TRY IT OUT

Computers are structured in a directory hierarchy where directories can contain other directories or files.

- (directory is just another term for folder)

ls command

Lists the files and directories

TRY IT OUT

ls command

We can add arguments to the `ls` command.

Run `ls` with the output of the `pwd` command.

When you run `ls` with the path to your working directory, what do you see? Is it different from when you ran `ls`? Why or why not?

`ls` also can take the argument `-l`. What does this do?

Piping (|)

We can pass the output from one command as an argument to another command by using the `|` symbol.

TODO: Run the following command

```
pwd | ls
```

Do you see anything different than before when you just ran `ls`? Why or why not?

Home directory

So far we've used `ls` to look at the files and folders in our **home directory**.

A home directory is where all the files and folders that belong to a specific user exists.

`~` represents the home directory

Absolute vs Relative Paths

So far we used `ls` to look at files in our own home directory.

When we run the command `ls /home/<USERNAME>`, the argument was an **absolute path**.

- An absolute path is the path from the root (or top) of the computer/file system.
- We can also specify **relative paths**. These are paths that go from the current working directory to another folder that we specify.

Exercise: If we wanted to see what files exist in the Downloads directory using an absolute path, what argument would we pass to `ls`? Run this.

Now try it with a relative path

cd command

- stands for change directory.
- cd takes an argument: the directory to move to
- We can also run cd without any arguments.
 - Try it!
 - what directory do you think we are now in?
 - Home directory!

mkdir command

- We can create new directories in the command line using the command `mkdir`.
- `mkdir` takes an argument: the name of the directory to create
- See what happens if you try to run it without any arguments.
 - The error message should also tell you an option or flag to use with the `mkdir` that can help you figure out how to use `mkdir`.
- Now, make a directory for `cs367`
- How can we verify that it has been created?

Setting up a 367 directory

Using mkdir, cd, and ls, create the following folders and subfolders in cs367/. Tabs indicate that a folder is within another folder.

```
cs367/  
  hws/  
    hw01/  
    hw02/  
    ....  
  notes/  
    lec01/  
    ...  
    lec04/
```

touch command

- Creates a new file
- Takes one argument: the name of the new file
- Create a new file in cs367/lec04/ notes named test.txt
- Use ls to verify it has been created

cp command

- Copy command
- `cp` takes two arguments: the source (file to copy) and the destination (where to put the copy)
- Copy your test.txt file into lec03/ notes
 - Use ls to verify it is there
- Now, let's try to copy a directory!
 - What happens when you try to copy lec04/ notes to your home directory?

mv command

- Move command
- `mv` takes two arguments: the source (file to move) and the destination (where to move it to)
- Can also be used to rename
- Use move to rename your test.txt file to hello.txt

rm command

- rm command
- `rm` takes one argument: the file to remove
- Let's delete the test.txt file in lec03 notes
- Now, let's try to delete the lec03 folder all together.
 - What happens?

Editing Text

- Vim: an efficient and lightweight text editor
- Vim operates in two modes:
 - insert mode: keystrokes are interpreted as inserts into the file contents at the point of the cursor.
 - command or escape mode: keystrokes are interpreted as vim commands, which allow a user to do such things as saving, exiting, searching, or moving around in the file.
- To switch from insert mode to command mode press the ESC key
- To switch from command mode to insert mode press i
- To save the file and quit, enter command mode and run :wq
- To quit without saving, enter command mode and run :q!
- **Todo:** Open hello.txt, go into insert mode, add your name, switch to command mode, save the file and quit
- For more help, run vimtutor

Downloading and Viewing Files

- `wget` command - downloads files from the internet
- Takes one argument: the url of the file to download
- **TODO:** Download the competency exam code from lec01 using `wget`
 - Hint: right click and select copy link as
- Use `ls` to verify you successfully downloaded it
- Unzip it using **`unzip`** command
- Using `ls` and `cd`, explore the contents and file structure
 - What is it?

Jar files

- A bundle of .class files
- `jar tf FILE.JAR`
 - Lists the contents
- Can also be added to a classpath

javac command

- Compiles a java source file (.java) into bytecode (.class)
- Takes one argument: the file to compile
- -cp (or -classpath) is how you tell javac where to find classes outside your own source. Entries are separated by : (colon)
- **TODO:** Try to compile src/Stats.java and test/StatsTest.java

java command

- Takes one argument: the name of the class to execute
- -cp (or -classpath) is how you tell java where to find classes outside your own source. Entries are separated by : (colon)
- **TODO:** Try to run StatsTest

javap command

- Tool provided with the JDK (Java Developer Kit)
- Runs on the class file (binary) to give you the information that is stored there in human readable format

```
javap -c ClassName
```

Java Bytecode Part 2

Java Bytecode

- low-level, platform-independent set of instructions that the Java Virtual Machine (JVM) executes
- **Intermediate Representation**
- How is it different from the source?
 - Optimized (constant folding)
 - Not human readable

Warmup: Worksheet Exercise 2

Exercise 3

- Download Mystery.class from today's reading
- Run javap to inspect the bytecode
- **Exercise:** Work through it to figure out:
 - What does this code do?
 - What do you think the source looked like?

Method Calls

```
public static void main(String[] args) {
```

```
    int a = 1;
```

```
    int b = 2;
```

```
    int c = calc(a, b);
```

```
}
```

```
static int calc(int a, int b) {
```

```
    return (int) Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
```

```
}
```

```
public static void main(java.lang.String[]);
```

```
    0: iconst_1
```

```
    1: istore_1
```

```
    2: iconst_2
```

```
    3: istore_2
```

```
    4: iload_1
```

```
    5: iload_2
```

```
    6: invokestatic #2    // calc:(II)I
```

```
    9: istore_3
```

```
   10: return
```

```
static int calc(int, int);
```

```
    0: iload_0
```

```
    1: i2d
```

```
    2: ldc2_w        #3      // double 2.0d
```

```
    5: invokestatic  #5      // java/lang/Math.pow: (DD)D
```

```
    8: iload_1
```

```
    9: i2d
```

```
   10: ldc2_w        #3      // double 2.0d
```

```
   13: invokestatic  #5      // java/lang/Math.pow: (DD)D
```

```
   16: dadd
```

```
   17: invokestatic  #6      // java/lang/Math.sqrt: (D)D
```

```
   20: d2i
```

```
   21: ireturn
```

Constant Pool

Table that stores various constants needed for execution

Includes:

- Literals
- Symbolic references: accessing an object in memory using its name (or identifier) rather than its direct memory address

`invokestatic #2` executes the static method at the second index of the constant pool

Number sign refers to the constant pool

Method Calls

```
public static void main(String[] args) {
```

```
    int a = 1;
```

```
    int b = 2;
```

```
    int c = calc(a, b);
```

```
}
```

```
static int calc(int a, int b) {
```

```
    return (int) Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
```

```
}
```

```
public static void main(java.lang.String[]);
```

```
    0: iconst_1
```

```
    1: istore_1
```

```
    2: iconst_2
```

```
    3: istore_2
```

```
    4: iload_1
```

```
    5: iload_2
```

```
    6: invokestatic #2    // calc:(II)I
```

```
    9: istore_3
```

```
   10: return
```

```
static int calc(int, int);
```

```
    0: iload_0
```

```
    1: i2d
```

```
    2: ldc2_w           #3      // double 2.0d
```

```
    5: invokestatic    #5      // java/lang/Math.pow: (DD)D
```

```
    8: iload_1
```

```
    9: i2d
```

```
   10: ldc2_w           #3      // double 2.0d
```

```
   13: invokestatic    #5      // java/lang/Math.pow: (DD)D
```

```
   16: dadd
```

```
   17: invokestatic    #6      // java/lang/Math.sqrt: (D)D
```

```
   20: d2i
```

```
   21: ireturn
```

Non-static method calls

Method calls:

`invokevirtual` instead of `invokestatic`

Constructor calls (object creations):

`invokespecial`

Method Calls - nonstatic

```
class Example {  
    public void greet() {  
        System.out.println("hello");  
    }  
}  
  
Example e = new Example();  
example.greet();
```

```
0: new          #2          // Create a new Example object  
3: dup          // Duplicate the reference to the object on the operand stack  
4: invokespecial #3          // Call constructor (Example.<init>)  
7: aload_1      // Load the reference to the Example object (example)  
8: invokevirtual #4          // Call greet() on the instance of Example  
11: return
```


Given this source, what is the bytecode?

```
public class CountdownTimer {
    public static void main(String[] args) {
        CountdownTimer timer = new CountdownTimer();
        timer.startCountdown(10);
    }
    public void startCountdown(int start) {
        int count = start;
        while (count > 0) {
            System.out.println(count);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.err.println("Timer interrupted.");
            }
            count--;
        }
        System.out.println("Blast off!");
    }
}
```

```
public void startCountdown(int);
```

Code:

```
0: iload_1
1: istore_2
2: iload_2
3: ifle 22

6: getstatic #5 // Field java/lang/System.out:Ljava/io/PrintStream;
9: iload_2
10: invokevirtual #6 // Method java/io/PrintStream.println:(I)V
13: invokestatic #7 // Method java/lang/Thread.sleep:(J)V
16: iinc 2, -1
19: goto 2

22: getstatic #5 // Field java/lang/System.out:Ljava/io/PrintStream;
25: ldc #8 // String Blast off!
27: invokevirtual #6 // Method java/io/PrintStream.println:(Ljava/lang/String;)V
30: return
```

Given this source, what is the bytecode?

```
public class CountdownTimer {
    public static void main(String[] args) {
        CountdownTimer timer = new CountdownTimer();
        timer.startCountdown(10);
    }
    public void startCountdown(int start) {
        int count = start;
        while (count > 0) {
            System.out.println(count);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.err.println("Timer interrupted.");
            }
            count--;
        }
        System.out.println("Blast off!");
    }
}
```

```
public CountdownTimer();
```

Code:

```
0: aload_0
```

```
1: invokespecial #1 // Method java/lang/Object."<init>":()V
```

```
4: return
```

```
public static void main(java.lang.String[]);
```

Code:

```
0: new #2 // class CountdownTimer
```

```
3: dup
```

```
4: invokespecial #3 // Method "<init>":()V
```

```
7: astore_1
```

```
8: aload_1
```

```
9: bipush 10
```

```
11: invokevirtual #4 // Method startCountdown:(I)V
```

```
14: return
```

Method Descriptors

- Specify the method's return type and parameter types
 - At a lower level than java source code
 - Notably does not include the method name
-
- `()I` – Describes a method that takes no parameters and returns an `int`.

Method Descriptors

1. Basic Types:

- `B = byte`
- `C = char`
- `D = double`
- `F = float`
- `I = int`
- `J = long`
- `L<classname>;` = an object of a class (e.g., `Ljava/lang/String;` for `String`)
- `S = short`
- `Z = boolean`

2. Void Return Type:

- `V` is used for methods that return `void`.

3. Array Types:

- For arrays, you use `[]` followed by the descriptor for the element type (e.g., `[I` for an array of `int`).

Examples:

Method with one **String** parameter and **void** return type:

- `(Ljava/lang/String;)V` – A method that takes a **String** and returns **void**.

Method with two parameters: **int** and **double**, and returns **boolean**:

- `(ID)Z` – A method that takes an **int** and a **double**, and returns a **boolean**.

Method with an array of **int** and a **String** parameter, returning **void**:

- `([ILjava/lang/String;)V` – A method that takes an array of **int** and a **String**, and returns **void**.

Exercise 4

- Download Mystery2.class from today's reading
- Run javap to inspect the bytecode
- **Exercise:** Work through it to figure out:
 - What does this code do?
 - What do you think the source looked like?

Summary

- Java Bytecode Part II
 - Method Descriptors
- Unix Commands
- HW1 due Friday
- HW2 deadline will be extended